

WiFi Vision

Project Team

Muhammad Mujtaba	21L-6058
Junaid Khalid	21I-1142
Faiq Hayat	20I-2608

Session 2021-2025

Supervised by

Dr. Zeshan Khan



Department of Software Engineering

National University of Computer and Emerging Sciences
Islamabad, Pakistan

June, 2024

Contents

1	Introduction	1
1.1	Problem Statement	1
1.1.1	Expensive Hardware and Installation	1
1.1.2	Scalability Challenges	1
1.1.3	Vulnerability to Adversarial Environmental Conditions	2
1.1.4	Data Security Risks	2
1.1.5	High Energy Consumption	2
1.1.6	No Support for Law Enforcement or Emergency Services	2
1.2	Scope	2
1.2.1	Channel State Information IoT Recorder:	2
1.2.2	Long Short-Term Model with Flask API:	3
1.2.3	Responsive Web Application:	3
1.2.4	Interactive Mobile Application:	3
1.2.5	Integration With Emergency Services:	3
1.3	Modules	3
1.3.1	Channel State Information IoT Recorder	3
1.3.2	Long Short-Term Memory with Flask API	4
1.3.3	Web Application in ReactJS	4
1.3.4	Mobile Application in Flutter	5
1.4	User Classes and Characteristics	5
2	Project Requirements	6
2.1	Use-case/Event Response Table/Storyboarding	6
2.2	Functional Requirements	7
2.2.1	Channel State Information IoT Recorder	7
2.2.2	Long Short-Term Memory with Flask API	7
2.2.3	Web Application in ReactJS	7
2.2.4	Mobile Application in Flutter	8
2.3	Non-Functional Requirements	8
2.3.1	Reliability	8
2.3.2	Scalability	9
2.3.3	Usability	9

2.3.4 Performance	9
2.3.5 Maintainability	9
2.3.6 Availability	10
2.3.7 Security	10
2.3.8 Interoperability	10
2.4 Domain Model	10
3 System Overview	11
3.1 Architectural Design	11
3.2 Design Models	12
3.3 Data Design	14
References	16

List of Figures

Figure 1: Use Case Diagram	6
Figure 2: Domain Model Diagram.....	10
Figure 3: Architectural Diagram.....	11
Figure 4: Activity Diagram.....	11
Figure 5: Class Diagram.....	12
Figure 6: Sequence Diagram	12
Figure 7 State Transition Diagram	13
Figure 8 Data Design Diagram.....	15

Chapter 1

Introduction

Wi-Fi Vision or Wi-Vi is a smart AI-based surveillance system that combines Wi-Fi signals strength data with deep learning techniques to provide monitoring in real time. It utilizes Channel State Information (CSI) values of the wireless communication link, e.g. Wi-Fi, which are recorded by ESP32 NodeMCU SoC placed at certain points throughout the area under surveillance. By analyzing the variations in these readings, the system will detect and classify movements offering an innovative solution to costly traditional CCTV surveillance systems.

1.1 Problem Statement

The demand for surveillance systems is on a steady rise, highlighting the inherent limitations and challenges of these systems.

1.1.1 Expensive Hardware and Installation

Existing systems incur significant installation and maintenance expenses. Consequently, many users have no access to these systems, making them experience a sense of insecurity.

1.1.2 Scalability Challenges

These systems are restricted to the specific areas where the hardware is installed. This affects their ability to cover large or multiple spaces, therefore, they have scalability issues.

1.1.3 Vulnerability to Adversarial Environmental Conditions

Traditional systems operate under optimum environmental conditions, including adequate lighting conditions and unobstructed field of vision. However, with variations in environmental conditions, their performance is significantly impacted.

1.1.4 Data Security Risks

There is a risk of data footage corruption and, deletion, which can compromise the working of the system leading towards decreased or ineffective surveillance in critical situations.

1.1.5 High Energy Consumption

Current systems have high energy consumption. CCTV cameras and their control unit needs to operational 24/7 to provide surveillance, resulting in high energy consumption and costs.

1.1.6 No Support for Law Enforcement or Emergency Services

Existing systems have no integrated functionalities to alert emergency or law enforcement services in real time during security incidents, resulting in delayed response.

1.2 Scope

The scope of the Wi-Fi Vision encompasses the development of cost-effective and efficient solution that aims to utilize the Wi-Fi signals and deep learning to predict and monitor the presence of humans and other objects. The system eliminates the need for costly hardware and improves the overall experience in surveillance and monitoring. The project scope includes:

1.2.1 Channel State Information IoT Recorder:

Developed using ESP32 NodeMCU SoC and an embedded small OLED screen to display Wi-Fi Channel State Information (CSI) values in real-time.

1.2.2 Long Short-Term Model with Flask API:

Implemented in python using TensorFlow and trained on existing Channel State Information (CSI) values. This model will be deployed on a cloud service and will be exposed by Flask API endpoints.

1.2.3 Responsive Web Application:

Developed in ReactJS to provide a responsive interface to allow users to interact with their Wi-Fi system installed in their spaces.

1.2.4 Interactive Mobile Application:

Developed in Flutter to provide an interactive interface to allow users to perform the same operations as on their web application. Moreover, it will push alert notifications to users in case of any incident in real-time for more prompt response.

1.2.5 Integration with Emergency Services:

Wi-Fi Vision will provide with integration functionality with Emergency or Law Enforcement Agencies to enable prompt response in case of security incident.

1.3 Modules

Wi-Fi Vision extends to multiple modules that are critical to seamless operation and overall success of the project. Here is the list of modules of Wi-Fi Vision.

1.3.1 Channel State Information IoT Recorder

CSI IoT Recorder developed using ESP32 NodeMCU SoC and small OLED display to capture and display the Wi-Fi Signal Channel State Information (CSI) values.

1. Wi-Fi Signal Strength Recorder: Record Wi-Fi signal Channel State Information (CSI) values.
2. Energy Efficient: It requires less energy.
3. Integration with Flask API: Transmits Channel State Information (CSI) readings at designated time intervals to deployed model exposed by Flask API endpoint.

1.3.2 Long Short-Term Memory with Flask API

Long Short-Term Memory Deep Learning Model deployed on Cloud service e.g. AWS or Azure and exposed through API endpoints developed using Flask.

1. Monitoring: Analyze and interpret Channel State Information (CSI) data to detect human presence and object movement.
2. Decision Making: Provide predictions and make pertinent decisions to secure the area.

1.3.3 Web Application in ReactJS

Responsive and robust web application for ease of access of user.

1. Secure Access: User authentication and data protection through available security measures.
2. Real-Time Monitoring: Display of live demonstrations of detected human presence, actions and objects.
3. Dashboard Interface: A Dashboard responsible for displaying useful information like system status and other analytics.
4. System Configuration: Configure and adjust system settings, including sensitivity and alert thresholds.
5. Historical Data Review: Display of previous data to allow user have insights on nature of any event.
6. User Management: Allow for different user roles and access levels within the application.
7. Integration with Emergency Services: Provide support for integration with local law-enforcement or emergency services.

1.3.4 Mobile Application in Flutter

Interactive and robust mobile application for ease of access of user.

1. Real-Time Monitoring: Display of live demonstrations of detected human presence, actions and objects.
2. Push Notifications: Receive instant alerts and notifications about detected anomalies or security incidents.
3. System Configuration: Configure and adjust system settings, including sensitivity and alert thresholds.
4. Event History: Display of previous data to allow user have insights on nature of any event.
5. Offline Access: View cached data and receive notifications even when the mobile device is offline or in low connectivity areas.
6. Customized Alerts: Set up and customize alert and notification settings specific to the user's needs.
7. Secure Access: User authentication and data protection through available security measures.

1.4 User Classes and Characteristics

Various user classes and their characteristics for Wi-Fi Vision are listed below: -

User class	Description
Homeowners	Homeowners are the primary user of the Wi-Fi Vision, specifically using it to monitor their spaces. There can be potentially more users as this is a cost effective solution with around 60% of daily interaction with it for effective surveillance. An estimated 80% of the use of the system will come from mobile devices and remaining coming from web application.
Organizations	Organizations will use Wi-Fi Vision to monitor their business premises. There are an estimated of 150 organizations adopting this technology with their main use coming from web application.
Emergency or Law Enforcement Agencies	Law Enforcement agencies will interact with Wi-Fi Vision in case of alert during a security incident. They are expected to use this system through provided integration functionality on their desktop interfaces.

Chapter 2

Project Requirements

This chapter describes the functional and non-functional requirements of the project.

2.1 Use Case

The primary use case of Wi-Fi Vision is to provide with surveillance of the spaces, user wants to monitor. Its primary objective is to detect the presence of humans and their activities along other objects in the space.

Here is Use Case diagram for the Wi-Fi Vision: -

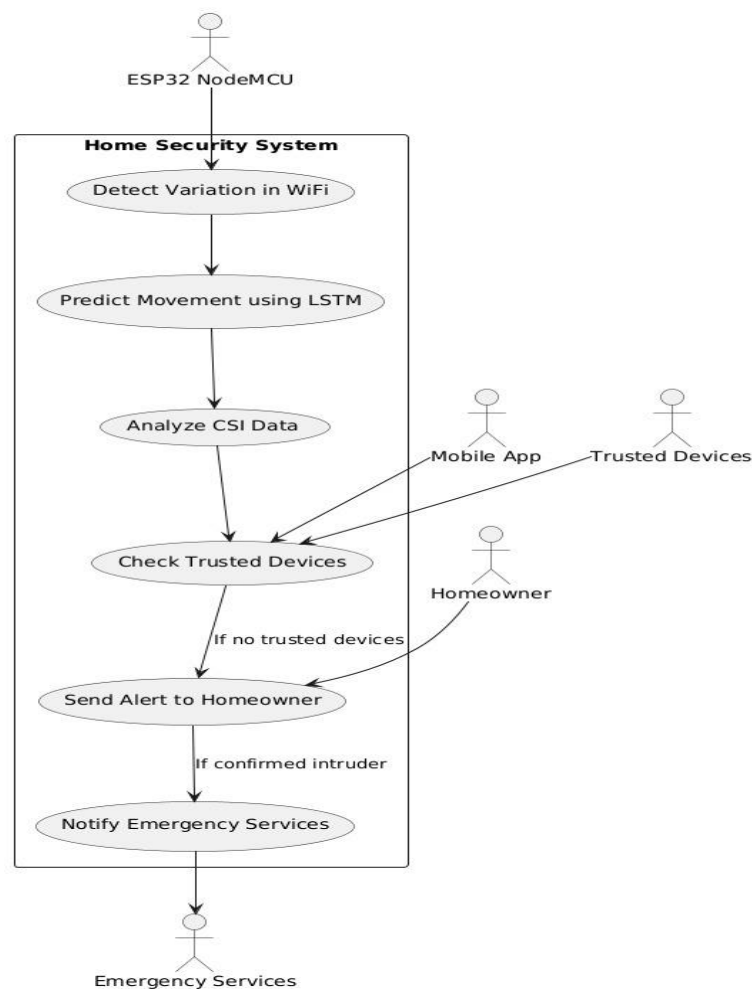


Figure 1: Use Case Diagram

2.2 Functional Requirements

This section contains the requirements for the Wi-Fi Vision. These requirements are organized by the scope of the project discussed in the proposal document.

2.2.1 Channel State Information IoT Recorder

Following are the requirements for Channel State Information IoT Recorder:

1. The system shall record Wi-Fi Channel State Information (CSI) values after a specific time interval.
2. The system shall transmit Channel State Information (CSI) data to Flask API endpoint after a specific time interval.

2.2.2 Long Short-Term Memory with Flask API

Following are the requirements for Long Short-Term Memory with Flask API:

1. The model shall communicate with ESP32 through Flask API.
2. The model shall process Channel State Information (CSI) data in real-time.
3. The model shall detect variations in Channel State Information (CSI) data.
4. The model shall classify the presence human activity or objects based on the variations of Channel State Information (CSI) data.
5. The model shall predict for each event with a confidence score.
6. The model shall trigger an event in case of detected anomalies.

2.2.3 Web Application in ReactJS

Following are the requirements for Web Application in ReactJS:

1. The system shall have an interactive and robust user interface.
2. The system shall allow users to securely login to access their security system.
3. The system shall allow users to upload the architectural plan of their premises.

4. The system shall display real-time human monitoring and object detection data.
5. The system shall provide heatmaps for visualization.
6. The system shall provide with a dashboard to display system status and analytics.
7. The system shall allow users to configure their system settings based on their preferences.
8. The system shall allow users to access surveillance data for the previous two days period.
9. The system shall provide a role-based access control to users for permissions.
10. The system shall provide with an option for integration with emergency services.

2.2.4 Mobile Application in Flutter

Following are the requirements for Mobile Application in Flutter:

1. The system shall allow users to securely login to access their security system.
2. The system shall send push notifications in case of security incident.
3. The system shall display real-time human monitoring and object detection data.
4. The system shall provide heat maps for visualization.
5. The system shall provide with a dashboard to display system status and analytics.
6. The system shall allow users to configure their system settings based on their preferences.
7. The system shall provide users with cached surveillance data to access it even when offline.
8. The system shall provide a role-based access control to users for permissions.
9. The system shall provide with an option for integration with emergency services.

2.3 Non-Functional Requirements

This section specifies nonfunctional requirements. These requirements are organized

by the features of the project discussed in the proposal document.

2.3.1 Reliability

The system shall maintain a reliable architecture with a 99.9pc up-time. The system due to its nature shall provide surveillance continuously regardless of environmental conditions. It shall be robust to handle any unexpected failures or errors.

2.3.2 Scalability

The system shall be scalable to monitor larger spaces upto the transmission range of the Wi-Fi signals. The system shall provide with integration of multiple Channel State Information (CSI) IoT recorders to cover more larger areas and provide a better accuracy.

2.3.3 Usability

The system shall provide with usable and interactive web and mobile applications to per-form related tasks. It shall provide a visual representation of monitored data. The system shall be simple to use and require minimum technical expertise to operate. The mobile app shall have support for offline usage for continuous monitoring.

2.3.4 Performance

The system shall meet real-time requirements for monitoring. It shall record Channel State Information (CSI) values at a regular interval to enable for real time monitoring. The system shall provide minimal latency to provide near instant feedback to the users. The model shall have a simple input to process data and predict the pattern, indicating presence of humans or objects, in the data to provide feedback to the user.

2.3.5 Maintainability

The system shall be developed using industry line development techniques to provide with a modular architecture. The system shall be documented properly to provide with easy to maintain code bases across all modules. The applications shall be containerized to maintain them.

2.3.6 Availability

The system shall be available all the time with maximum uptime. The Flask API, web and mobile applications shall be deployed on cloud to ensure system operationality.

2.3.7 Security

The system shall employ industry line security to provide with secure and easy to use. User authentication shall utilize JsonWebTokens (JWT) for secure login. The system shall leverage Role-Based Access Control to only allow authorized users to view and manipulate surveillance data.

2.3.8 Interoperability

The system shall support integration with external systems like emergency services for alerts during security incident. The system shall allow for use cases beyond surveillance like telecommunications optimization, etc.

2.4 Domain Model

Here is the domain model diagram for Wi-Fi Vision: -

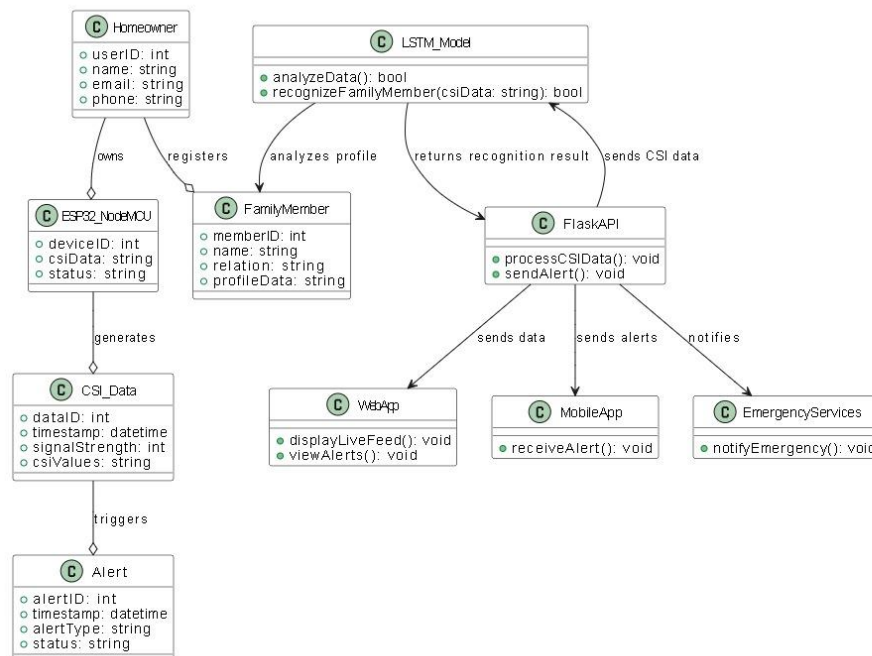


Figure 2: Domain Model Diagram

Chapter 3

System Overview

Give a general description of the functionality, context, and design of your project.
Provide any background information if necessary.

3.1 Architectural Design

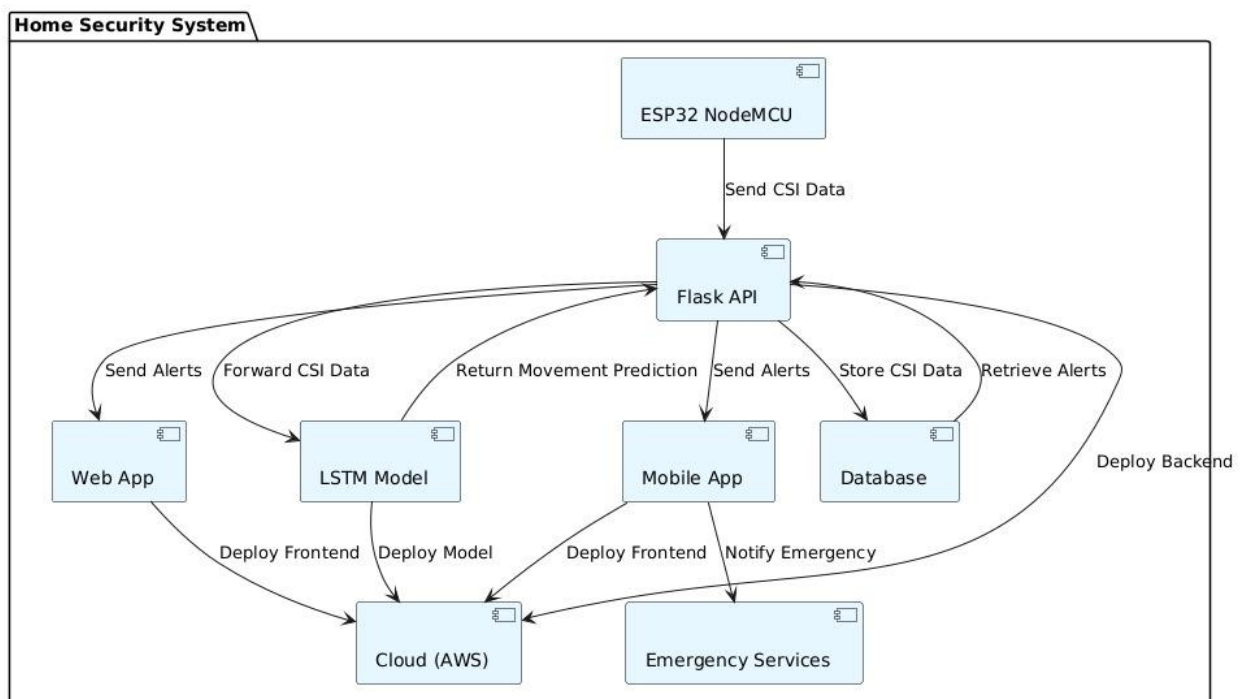


Figure 3: Architectural Diagram

3.2 Design Models

Here are the design models for Wi-Fi Vision: -

- **Activity Diagram**

This diagram depicts the flow of activities in the Wi-Fi Vision, displaying the sequence of tasks in detecting human presence and object detection. Firstly, ESP32 records variations in Channel State Information (CSI) values. Then, this data is transmitted to LSTM model exposed via Flask API. Model makes predictions based on variations in Channel State Information (CSI) values. Then, result is displayed on web and mobile applications.

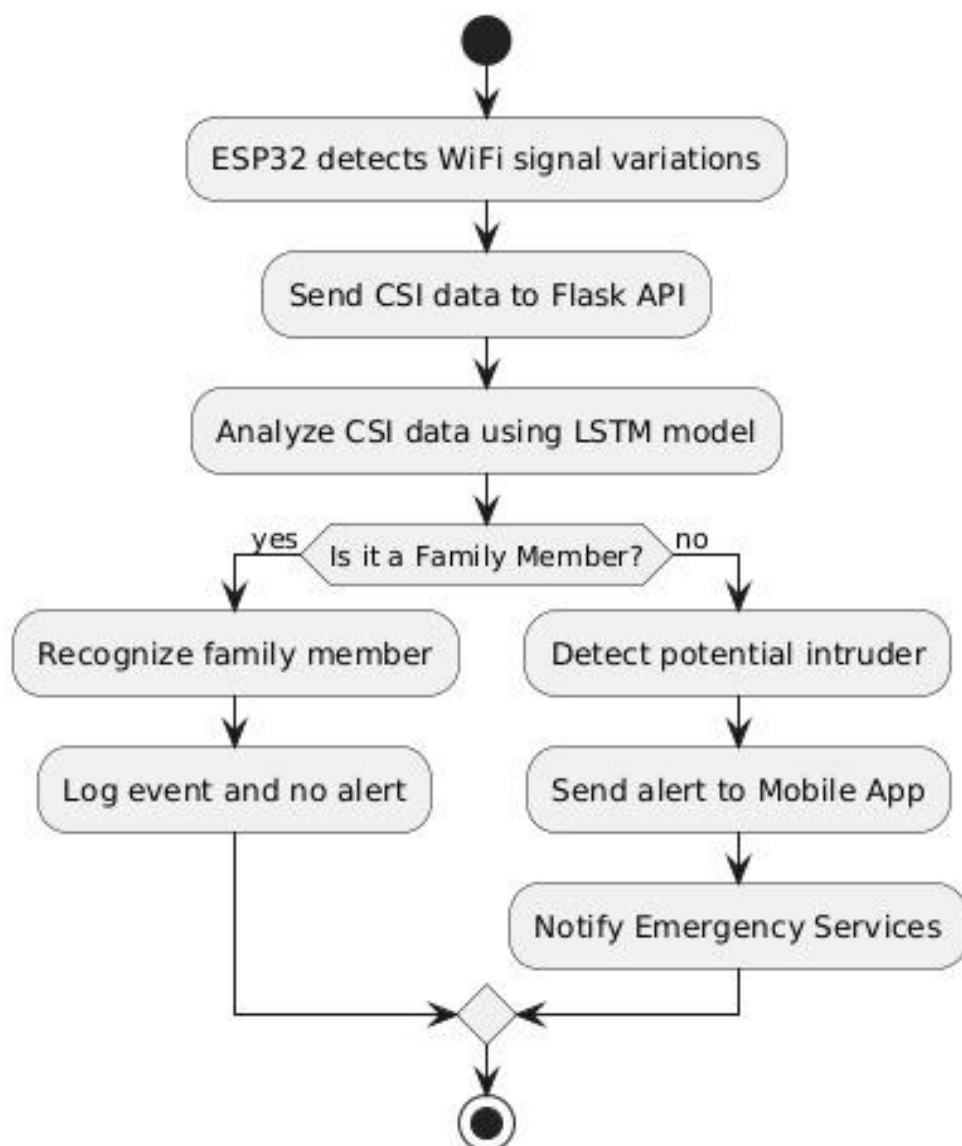


Figure 4: Activity Diagram

• Class Diagram

This diagram shows various classes and their relationships in the Wi-Fi Vision system. It shows the structure of the system at code level, outlining main modules and their attributes, methods, and relationships with each other.

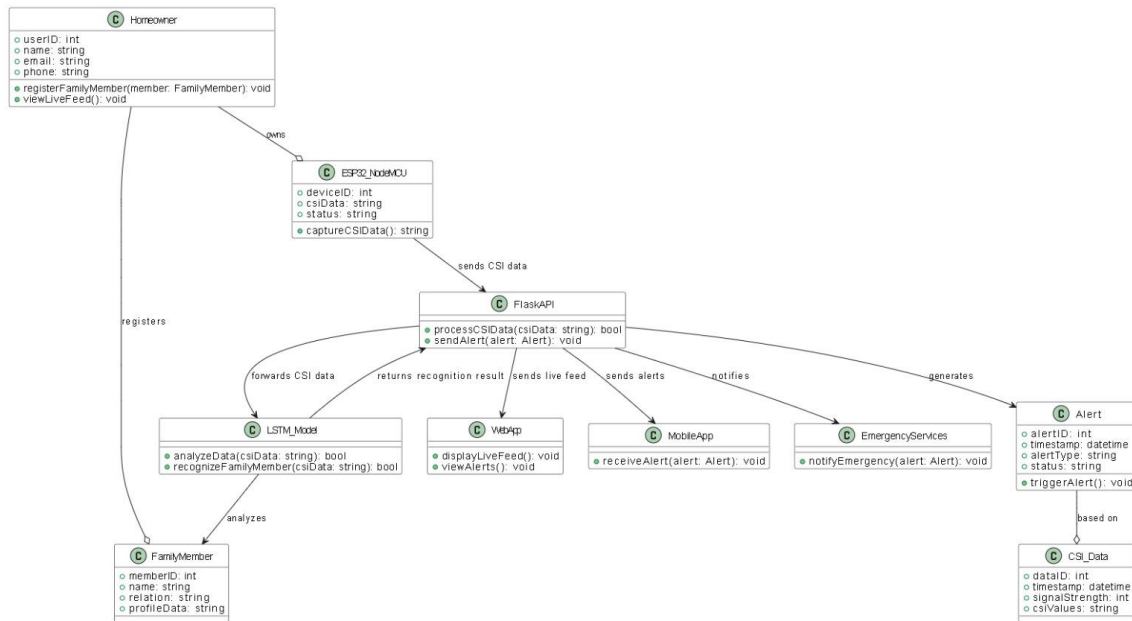


Figure 5: Class Diagram

• Class-level Sequence Diagram

This diagram concentrates on the interactions between the classes in a specific use case. It shows how the system detects human activities and objects presence and then, sending an alert to the user. It shows the sequential flow of the system.

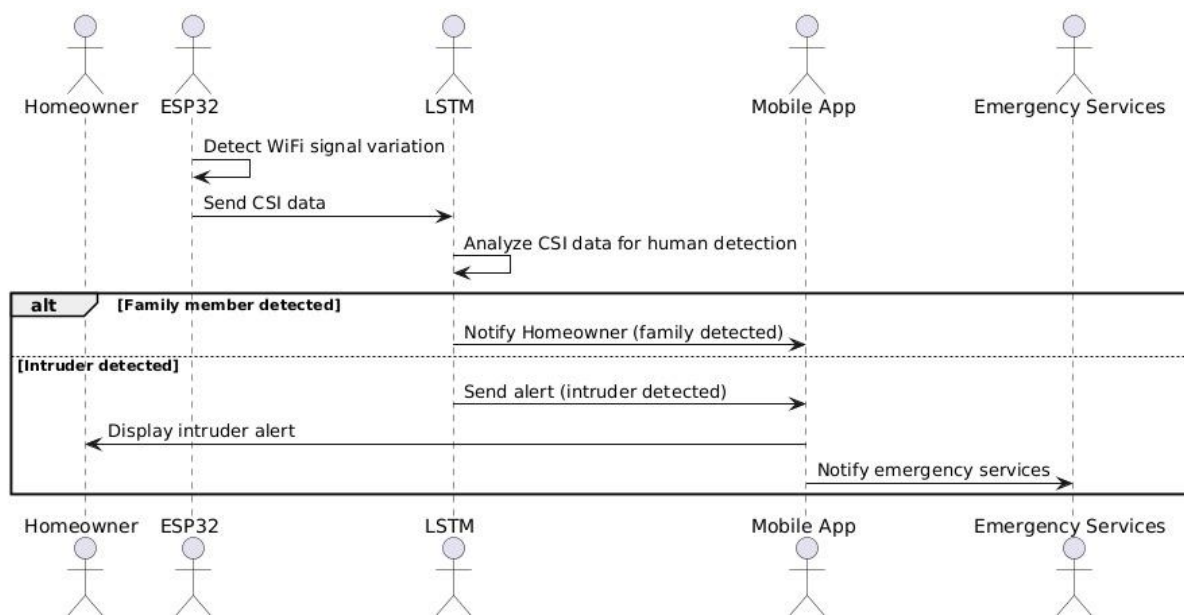


Figure 6: Sequence Diagram

- **State Transition Diagram**

This diagram exhibits different states that the Wi-Fi Vision system can be in. It shows how the system transitions from one state to another and between these states, events and conditions are also displayed.

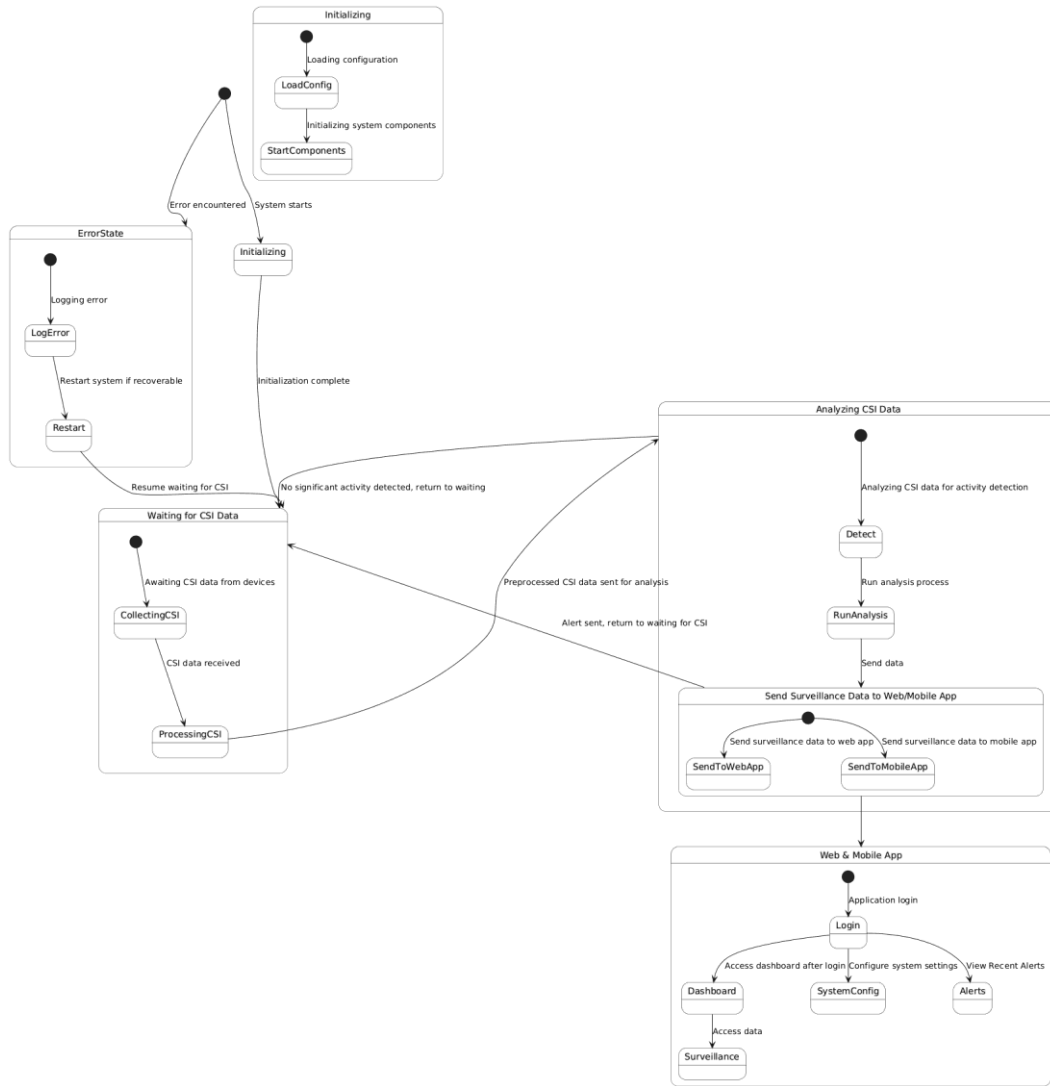


Figure 7 State Transition Diagram

3.3 Data Design

The data design diagram of Wi-Fi Vision system concentrates on structured data entities, organizing them for efficient processing.

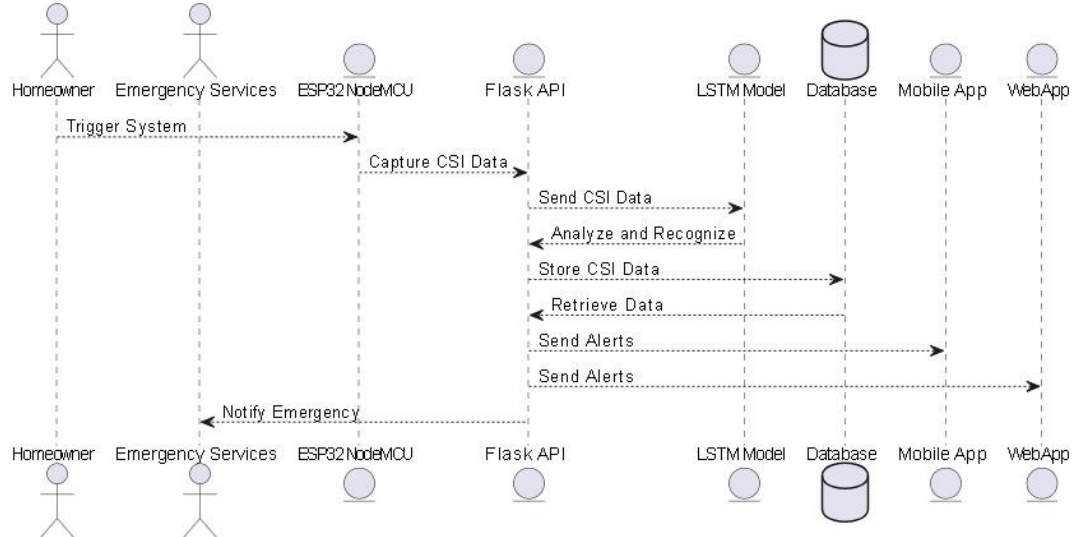


Figure 8 Data Design Diagram

Bibliography

- George Eisa Wilson Wu, C. D. (2017). *Pronto Vision*. Hack the North.
- Johnson., N. I. (2016). *A Review on Wi-Fi Sensing Technology*. Current Trends in Signal Processing.
- Muhammad Usman, A. Z. (2019). *Human Activity Recognition using Radio Waves*.
- Yang Hu Ying He, Y. C. (2020). *Wi-Fi Sensing Recognition with Commodity*. IEEE Internet of Things .
- Fabbro, D. G. (2023). *Sharp: Environment and Person Independent Activity Recognition with Commodity*. IEEE Transactions on Mobile Computing.